Manuscript prepared November 4, 2026, as part of the Fundamentals of Data Engineering for AI course project.

SkillSync AI: An Intelligent Data Engineering Platform for Real-Time Job Market and Emerging Skill Intelligence

SUDHAMSHA A, V AKSHAYA, AND TANISHKA

Guide: Dr. N. Shirisha, Associate Professor

Department of [Department Name], [Institution Name], Telangana, India (Roll Nos. 2420030753, 2420090087, 2420030619)

Corresponding author: Sudhamsha A ([email address])

This work was carried out as an academic project under the guidance of Dr. N. Shirisha. It received no external funding.

ABSTRACT Job postings pile up faster than anyone can read them, and the skills employers really care about sit inside free-text descriptions that look different from one source to the next. This paper presents SkillSync AI, a data engineering platform that turns such postings into skill-level intelligence. The design joins batch and streaming ingestion, automated validation and deduplication, PySpark-based processing, dictionary-driven NLP for skill extraction, and a Streamlit dashboard for demand analysis, emerging-skill spotting and resume-based skill-gap checks. We lay out the complete proposed architecture and then report on the working prototype, which we ran on a 40-posting sample covering ten technical roles in four Indian cities. The pipeline recovered 22 distinct skills, with Python appearing in 80% of the postings, and tied skill groups to average salaries between ₹5.7 lakh and ₹11.1 lakh per role. We are upfront about what the prototype does not do yet: streaming ingestion, warehouse storage and demand forecasting exist as design rather than code, and the sample is far too small for statistical claims. The paper closes with a gap analysis and a list of next steps.

INDEX TERMS Data engineering, job market analytics, natural language processing, PySpark, skill extraction, skill-gap analysis.

I. INTRODUCTION

Every day, employers push thousands of openings onto job portals, company career pages and recruiter feeds. Each source has its own idea of what a record looks like. One gives a tidy JSON object with a separate skills field, another gives a CSV row with a paragraph of prose, and a third sends little more than a title and a link. The detail that matters most to a job seeker or a curriculum designer, namely which tools and skills are being asked for, tends to hide in sentences such as "we are looking for someone comfortable with Python, SQL and AWS." A student trying to decide between learning Spark or Kubernetes first has no simple way to find out which one the market is rewarding. Annual labour surveys arrive too late to help, and reading postings by hand does not scale beyond a few dozen.

We see this mainly as a data engineering problem and only secondarily as a modelling one. Before any language model or classifier can say something useful, the records have to be collected from several formats, checked, stripped of duplicates and written somewhere they can be queried again. Zeydan and Mangues-Bafalluy describe data engineering in just these terms, as the design and upkeep of systems that take raw sources and produce dependable information for analytics and AI/ML [7]. Their six-module view of a platform (connect, ingest, analyse, store, monitor and visualise, manage and orchestrate) gave us a convenient skeleton, and we have organised much of this paper around it.

SkillSync AI is our attempt to put those pieces together for the job-market case. It takes raw postings in, cleans and standardises them with PySpark, pulls skills out of the description text, and serves the result through an interactive dashboard that also lets a user upload a resume and see which skills a target role would still ask of them. Part of the system is built and running. Part of it, such as Kafka streaming and the forecasting module, is so far a design. We try to keep that line visible throughout.

A. RELATED WORKS

Skill extraction from postings has attracted a good deal of attention lately. Senger *et al.* [1] surveyed deep-learning approaches to the task, catalogued the public datasets, and tried to settle the loose terminology around "hard" and "soft" skills. Their survey covered 26 neural-extraction publications up to late 2023 and deliberately left out counting and topic-model methods. Nguyen *et al.* [2] asked whether large language models can do the job with in-context learning instead of annotated training data. Across six harmonised datasets the LLMs did not beat supervised models overall, yet they coped better with syntactically awkward skill mentions.

A different thread compares how skills get categorised. Ao *et al.* [3] ran dictionary-based methods and LDA topic modelling over UK job advertisements and scored each by how much of the variation in wages its skill variables could explain. Dictionaries explained roughly a fifth of it, topic modelling considerably more, although the authors admit that the LDA topics are harder to interpret. We take two things from this: a dictionary is a reasonable, transparent starting point, and interpretability is worth protecting.

Other papers move closer to pipelines. Tzimas *et al.* [4] describe a procedure for turning online postings into labour-market intelligence, covering source selection, extraction, cleansing, normalisation and deduplication, and then apply it to Greek tourism vacancies during COVID-19 and to ontology-based skill extraction. Their method uses classical NLP and machine learning rather than LLMs. SKORE [5] sets up an LLM-driven framework with a human in the loop and four stages: extraction, normalisation, categorisation and counting. On the

economics side, Oleš [6] used Slovak online vacancies to study how in-demand skills, advertised wages and automation exposure move together, and found wage premia for machine-learning and AI skills.

Taken together, these studies are strong on one slice each: extraction accuracy, categorisation, a specific labour-market question. None of them, as far as we could tell, also gives the surrounding plumbing a first-class role, that is, ingestion, quality control, scalable storage, trend detection and a decision-facing interface in a single system. That gap is where SkillSync AI sits.

B. OVERVIEW AND PROJECT OBJECTIVES

Fig. 1 shows the proposed platform. Job data arrives from files, APIs and historical archives; Apache Kafka and a real-time cache form the ingestion layer, and raw data is kept in a MinIO/Parquet lake. PySpark processes the data and runs quality validation, and the cleaned result goes into a PostgreSQL warehouse. An AI skill-intelligence layer (NLP extraction, skill normalisation and ML forecasting) reads from the warehouse, and a decision layer of Streamlit and Power BI serves job seekers and recruiters. Apache Airflow is our intended scheduler for these stages. We set four objectives for the project:

- **Intelligent data integration.** Build a pipeline that brings heterogeneous job-market data from several sources into reliable, analytics-ready form.
- **Skill intelligence.** Extract, normalise and categorise skills from unstructured descriptions with NLP.
- **Emerging skills and forecasting.** Track changing demand, flag skills that look like they are rising, and eventually forecast demand.
- **Skill-gap and decision support.** Offer role-based skill-gap analysis through an interactive dashboard.

The contributions of this paper are modest and practical. We describe an end-to-end architecture that joins data engineering with skill NLP (Section VI). We report a working prototype with the code-level details of its ingestion, cleaning, extraction, analytics and dashboard stages (Sections II to V and VII). And we give an honest account of its results and its limits on a small sample (Sections VIII and IX).

The rest of the paper is organised as follows. Section II covers data sources and ingestion, Section III data quality and processing, and Section IV skill extraction. Section V explains the analytics engine, Section VI the architecture and technology stack, and Section VII the dashboard and resume-based gap analysis. Results are in Section VIII, gaps and future work in Section IX, and Section X concludes.

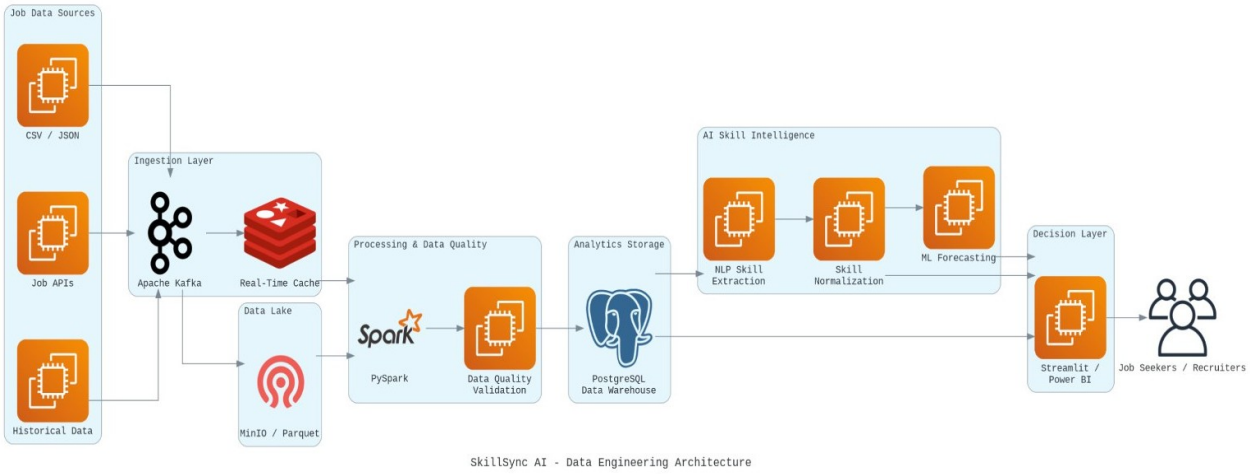


FIGURE 1. SkillSync AI data engineering architecture. Job data from files, APIs and historical archives enters through Kafka and a real-time cache, lands in a MinIO/Parquet data lake, is processed and validated with PySpark, stored in a PostgreSQL warehouse, enriched by the AI skill-intelligence layer and delivered through Streamlit or Power BI.

II. DATA CONNECTION AND INGESTION

A. DATA SOURCES

The architecture expects three kinds of input: flat files (CSV and JSON exports), pull-based APIs offered by job portals, and push-style event streams for postings as they appear. Historical files and live events are treated as two paths into the same lake, so a skill trend computed on last year's archive and one computed on this morning's postings come out of identical downstream code. For the streaming path we have chosen Apache Kafka [12], largely because it decouples the producers of postings from the Spark jobs that consume them.

In the current prototype only the batch CSV path exists. Scraping portals at scale raises licensing and terms-of-service questions that we have chosen not to take on within a course project, so the live connectors are still on the to-do list.

B. DATASET USED IN THE PROTOTYPE

All the numbers in this paper come from a small sample file, *jobs.csv*, with six columns. Table 1 summarises it. Ten technical roles appear, each posted once in each of four cities, so the file is perfectly balanced by role and by city. That balance is convenient for testing a pipeline, but it also means skill counts come out as multiples of four and should not be read as a picture of real market demand.

TABLE 1. Summary of the sample job-postings dataset used for the prototype.

Property	Value
Records	40 job postings
Fields	job_id, title, description, location, salary, posted_date
Distinct roles	10 (4 postings each)
Cities	Hyderabad, Bengaluru, Chennai, Pune (10 each)
Posting dates	5 Jan to 25 Aug 2026 (5 per month)
Salary range	₹5.50 to ₹11.50 lakh per year
Mean / median salary	₹8.83 / ₹9.25 lakh
Missing values	0 in all six fields
Duplicate job_id	0

C. INGESTION IMPLEMENTATION

Two small scripts handle loading. The first reads the CSV with pandas [10] and prints the row count and column names, which is a cheap sanity check we run before anything else. The second, the Spark job, reads the same file with the header option on and schema inference enabled. Running Spark in local mode with all cores uses every core on one laptop; moving to a cluster would only mean changing the master URL, not the transformation code.

III. DATA QUALITY AND PROCESSING

A. VALIDATION AND CLEANING RULES

The cleaning stage is deliberately plain. Records are de-duplicated on *job_id*; any row with a missing title or description is dropped, since without one of them there is nothing to extract skills from; leading and trailing whitespace is trimmed from both text fields; and a *processed_at* timestamp is stamped on every surviving row so that we can tell later which run produced it.

On our sample this pass removed nothing. All 40 identifiers were unique and no field had a missing value (Table 1). That is a fair result for a curated file, but it means the quality rules have so far been exercised only on clean data. We have not yet tested them on deliberately corrupted input.

A fuller design uses Great Expectations suites to express rules declaratively, for instance a non-null title, a numeric salary inside a plausible range, an ISO-formatted date and a unique identifier. A failing batch would then produce a validation report rather than silently losing rows.

B. DISTRIBUTED PROCESSING WITH PYSPARK

Spark was a natural choice. It keeps intermediate data in memory instead of writing it back to disk between stages, an improvement over the classic MapReduce model [9] and one of the reasons it became a general engine for batch and streaming analytics [8]. For forty rows the benefit is of course nil; we chose it so the same code keeps working when postings arrive by the million. We have not benchmarked throughput, and we deliberately make no speed claims.

C. STORAGE AND DATA FLOW

Cleaned records are written as Snappy-compressed Parquet files under *data/processed/jobs*. Parquet is columnar, so queries that touch only a few columns, such as “average salary by title,” skip the bulky description text. The raw file stays untouched in *data/raw*, which gives us a simple raw-versus-processed lake layout. The planned design puts these files on MinIO object storage and loads curated tables into PostgreSQL under a star schema, with posting facts joined to role, location, skill and date dimensions. Fig. 2 shows the sequence of stages.

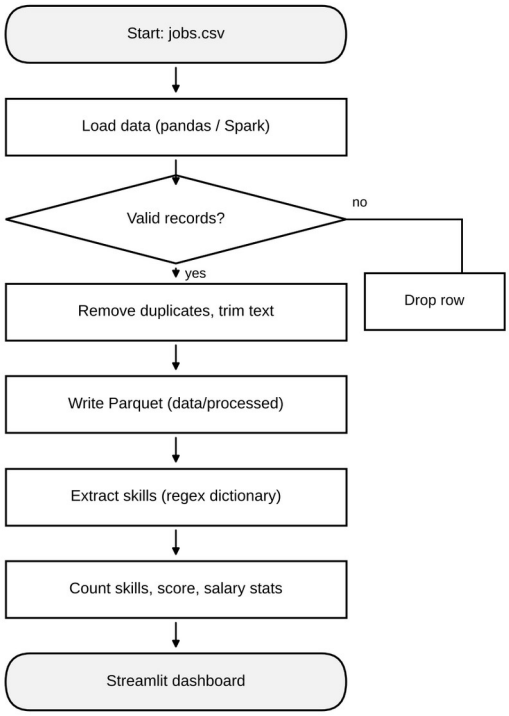


FIGURE 2. Flow of the implemented prototype pipeline, from the raw CSV file to the Streamlit dashboard.

IV. SKILL EXTRACTION

A. DICTIONARY-BASED EXTRACTION

Extraction starts from a curated list of 22 skills: Python, SQL, Pandas, NumPy, Git, PySpark, Hadoop, AWS, Machine Learning, TensorFlow, Excel, Power BI, Docker, Kubernetes, Linux, Deep Learning, PyTorch, NLP, Django, REST API, PostgreSQL and Scikit-learn. For each description the script tests every dictionary entry with a case-insensitive regular expression wrapped in word boundaries, with the skill name escaped so that characters like the hyphen in “Scikit-learn” are not read as pattern syntax. Matches are joined into one comma-separated string and saved next to the original row.

The word boundaries are not decoration. Without them, “Git” would fire inside “digital” and “NLP” inside any longer token that happens to contain it. Even so, the method is blunt. The word “Excel” can be a spreadsheet or a verb (“you will excel at…”), and a case-insensitive match cannot tell which. In our sample that never happened, because every description lists skills plainly, but on real postings it would produce false positives. Context-

sensitive disambiguation is therefore on our list of required improvements.

B. NORMALISATION AND CATEGORISATION

Because the dictionary stores one canonical spelling per skill, whatever form appears in the text is reported in the same way, which is a first, crude form of normalisation. A fuller alias table (for example mapping “sklearn” to Scikit-learn or “Postgres” to PostgreSQL) is planned, along with a category layer that groups skills into programming languages, data and big-data tools, cloud and DevOps, machine learning and AI, and business intelligence. These two steps mirror the normalise-then-categorise stages of SKORE [5] and the normalisation emphasis in [4].

C. BEYOND DICTIONARIES

A dictionary cannot find a skill that it has never been told about, and emerging skills are, by definition, the ones that are not yet on anybody’s list. The planned next step is to add spaCy-based entity recognition and, where budget permits, a transformer or LLM extractor. The literature suggests a trade-off to keep in mind: LLM prompting tolerates unusual phrasing but still lags supervised taggers on standard benchmarks and costs more to run [2], while topic models are flexible but harder to explain [3]. We expect a hybrid, with the dictionary as a fast, auditable first pass and a learned model for the leftovers.

V. SKILL INTELLIGENCE ENGINE

A. DEMAND ANALYSIS

Once skills are attached to postings, the analytics step is a simple count. A *Counter* tallies how many postings mention each skill, the result is sorted from most to least frequent and saved as *skill_frequency.csv*, and the dashboard recomputes the same counts on the fly for whichever location or role the user has filtered to.

B. EMERGING-SKILL DETECTION

The current heuristic is intentionally modest. A skill is flagged as a potential emerging skill if it shows up in at most two postings yet is tied to at least two different job roles. The reasoning is that a skill which is still rare but already crossing role boundaries may be spreading. It is a cross-sectional rule: it looks at one snapshot, not at change over time.

On the full sample the rule flags nothing, because the rarest skill appears in four postings. Filtering to a single city cuts every count to a quarter, and six skills (Pandas, Hadoop, Machine Learning, Docker, Kubernetes and Linux) then pass the test, each with two postings spread over two roles. Those are simply the skills that two roles happen to share, so the flag reflects the structure of the sample and not a rise in demand. We therefore treat the rule as a placeholder. A proper detector needs counts per time window and a growth measure, which the planned forecasting module would supply.

C. OPPORTUNITY SCORE

To rank skills for a learner, the dashboard combines current demand with breadth. For a skill k, let $d(k)$ be the number of postings that mention it and $r(k)$ the number of distinct roles that require it. The opportunity score is

$$S(k) = d(k) + r(k) \quad (1)$$

This is a heuristic rather than a statistical model. Its virtue is that anyone can reproduce it by hand and see why a skill ranks where it does. Its weakness is that it gives equal weight to a posting count and a role count, which live on different scales.

D. FORECASTING DESIGN

Demand forecasting is part of the proposed platform but is not implemented. The plan is to aggregate monthly posting counts per skill and fit simple baselines first, such as a moving average and a linear trend in scikit-learn [11], before trying heavier models. We also checked whether our sample could support even that. It spans eight months with exactly five postings in each, so the monthly volume is flat by construction, and any “forecast” from it would mean nothing. Forecasting will have to wait for a longer, real series.

VI. SYSTEM ARCHITECTURE AND TECHNOLOGY STACK

A. LAYERED ARCHITECTURE

Fig. 1 arranges the platform in layers. On the left, job data sources (CSV/JSON files, job APIs and historical data) feed the ingestion layer, where Kafka carries the events and a real-time cache keeps the freshest postings quickly reachable. Raw data is also written to the data lake as Parquet on MinIO. The processing and data-quality layer, built on PySpark, validates and transforms the data, and the curated result lands in a PostgreSQL warehouse that acts as analytics storage. The AI skill-intelligence layer then runs NLP skill extraction, skill normalisation and ML forecasting. Finally, a decision layer built with Streamlit and Power BI puts the results in front of job seekers and recruiters. The arrangement maps closely onto the six-module reference view of [7]. Airflow is not drawn in the figure, but it is the orchestrator we have in mind for scheduling every stage.

Only part of this picture exists as running code. The next subsection states which part.

B. TECHNOLOGY STACK AND IMPLEMENTATION STATUS

Table 2 lists the tools chosen for each layer and whether they are actually exercised by the current prototype. Our slides described the stack as a whole, and reading them back we felt that a reader of a paper deserves this finer distinction.

TABLE 2. Technology stack of SkillSync AI and implementation status of each component in the current prototype.

Layer	Tools	Status
Language	Python	Implemented
Processing	pandas, Apache Spark (PySpark)	Implemented
Storage	CSV, Parquet (pyarrow)	Implemented
	Real-time cache (Redis)	Planned
	MinIO, PostgreSQL star schema	Planned
Ingestion	Batch CSV	Implemented
	Kafka streaming, REST APIs	Planned
Data quality	Spark dedup, null drop, trim	Implemented
	Great Expectations	Planned

Layer	Tools	Status
NLP	Regex skill dictionary	Implemented
	spaCy, transformer / LLM	Planned
ML	scikit-learn forecasting	Planned
Orchestration	Apache Airflow	Planned
Interface	Streamlit, pypdf	Implemented
	Power BI	Planned
Deployment	Git; Docker	Git implemented; Docker planned

VII. DECISION LAYER: DASHBOARD AND RESUME SKILL-GAP ANALYSIS

A. INTERACTIVE DASHBOARD

The dashboard, written in Streamlit, loads the processed postings and the skill-frequency table. A sidebar offers location and role filters, and everything below recomputes when they change. At the top are three headline metrics (postings, distinct skills and the leading skill), followed by a browsable table of postings with formatted salary and date columns. Further down are a bar chart of the ten most requested skills, a salary section with average, highest and lowest figures plus average salary by role, a panel of potential emerging skills with their related roles, and the ranked opportunity table of Section V.

B. RESUME-BASED SKILL-GAP ANALYSIS

A user can upload a resume as a PDF. The text is pulled out page by page with *pypdf*, lower-cased, and matched against the skill dictionary using the same word-boundary approach as Section IV. If a specific role is selected, the app takes that role's required skills, splits them into matched and missing, and reports a match percentage:

$$M = |R \cap U| / |R| \times 100 \quad (2)$$

where R is the set of skills the role requires and U the set found in the resume. To make this concrete, a Data Engineer posting in our sample requires five skills (Python, SQL, PySpark, Hadoop, AWS). A resume that mentions only Python and SQL would score M = 2/5 = 40%, and the page would list PySpark, Hadoop and AWS as the gaps. That worked example is illustrative; we have not yet run a user study.

Two simplifications deserve mention. The role profile is taken from the first matching posting, which is fine here because all postings for a role share one description, but real data will need an aggregate profile across many postings. Also, the resume matcher can only see skills that are in the dictionary, so it inherits every limitation of Section IV.

VIII. RESULTS AND DISCUSSION

A. SKILL DEMAND

Running the extraction script over the 40 descriptions returned 22 distinct skills, every one of them from the dictionary, with an average of 4.1 skills per posting (28 postings yielded four skills, eight yielded five and four yielded three). Fig. 3 gives the full ranking. Python stands well clear at 32 postings, or 80% of the sample, and it is required by eight of the ten roles. AWS and SQL come next with 16 postings each (40%), each shared by four roles. Six skills (Pandas, Hadoop, Machine Learning, Docker, Kubernetes and Linux) appear in eight postings, and the remaining thirteen in four apiece.

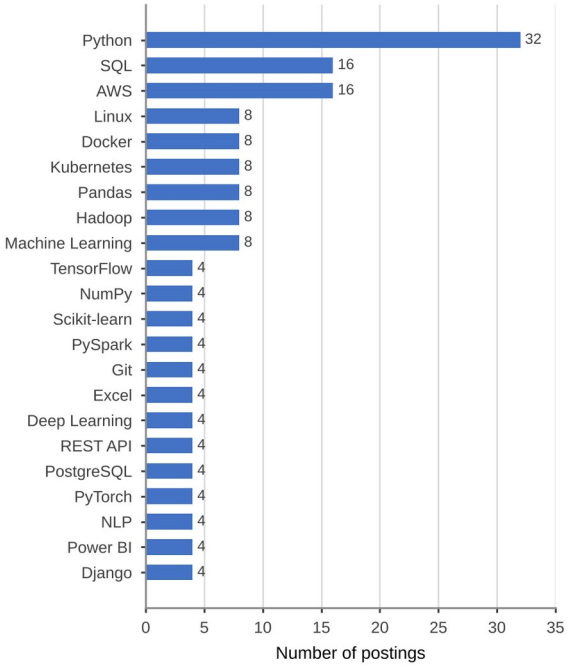


FIGURE 3. Skill demand across the 40 sample postings. Darker bars mark skills in at least 16 postings.

The opportunity score of Section V ranks Python first at 40 (32 postings plus 8 roles), then AWS and SQL at 20 each, the six eight-posting skills at 10, and the remaining skills at 5. Because our sample repeats each role description four times, this ordering is really a ranking of how many *roles* use a skill, so we would not lean on it until it is recomputed on real data.

B. ROLE-WISE REQUIREMENTS

TABLE 3. Skills extracted per role and mean advertised salary.

Role	Skills extracted	Mean salary (₹ lakh)
AI Engineer	Python, Deep Learning, PyTorch, NLP	11.1
Big Data Engineer	Python, Hadoop, AWS	10.6
ML Engineer	Python, SQL, Machine Learning, TensorFlow	10.1
Data Scientist	Python, Pandas, Machine Learning, Scikit-learn	9.6
DevOps Engineer	AWS, Docker, Kubernetes, Linux	9.6
Data Engineer	Python, SQL, PySpark, Hadoop, AWS	9.1
Cloud Engineer	AWS, Docker, Kubernetes, Linux	8.7
Backend Developer	Python, Django, REST API, PostgreSQL	7.6
Python Developer	Python, SQL, Pandas, NumPy, Git	6.1
Data Analyst	Python, SQL, Excel, Power BI	5.7

Table 3 shows that the roles separate sensibly. Cloud-oriented roles lean on AWS, Docker, Kubernetes and Linux, the data roles on Python, SQL and the Hadoop/Spark family, and the AI roles on deep-learning frameworks. One result is worth noticing: Cloud Engineer and DevOps Engineer were extracted with the identical four skills but differ by about ₹0.9 lakh in mean salary. Skill lists alone clearly do not explain pay, which fits the wage-regression perspective of [3] and suggests that seniority, employer and other fields would need to enter a richer model.

C. SALARY PATTERNS

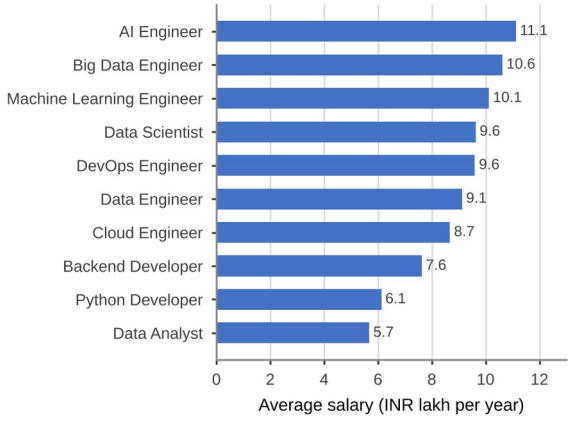


FIGURE 4. Average advertised annual salary by role in the sample (INR lakh).

Average salary runs from ₹5.7 lakh for Data Analyst to ₹11.1 lakh for AI Engineer (Fig. 4). Skills linked to the highest-paid roles were Deep Learning, NLP and PyTorch (₹11.1 lakh average), followed by Hadoop and Machine Learning (₹9.9 lakh), while Excel and Power BI sat at the bottom (₹5.7 lakh). This echoes, in a very small way, the premia that Oleš [6] reports for machine-learning skills. By city, Bengaluru averaged ₹9.2 lakh, Pune ₹8.9 lakh, Hyderabad ₹8.7 lakh and Chennai ₹8.5 lakh. With ten postings per city and the same ten roles in each, the spread is small and we would not read much into it. The skill mix was identical across all four cities.

D. COMPARISON WITH EXISTING APPROACHES

Table 4 contrasts the design with the studies reviewed in Section I-A. This is a qualitative comparison of scope, not a head-to-head benchmark: we did not run the other systems on our data, and several rows describe capabilities that are planned rather than built (Table 2).

TABLE 4. Qualitative comparison of typical existing work with the proposed platform.

Aspect	Typical existing work	SkillSync AI (as designed)
Data integration	Single or fragmented sources	Multi-source, batch plus streaming
Data quality	Limited automated validation	Validation, cleaning, deduplication
Skill extraction	Extraction is the main focus	Extraction, normalisation, categorisation
Scalability	Dataset-specific pipelines	Spark-based pipeline
Emerging skills	Mostly current patterns	Trend-based detection (planned)
Forecasting	Rarely tied to extraction	ML-based demand forecast (planned)
Skill-gap analysis	Seldom integrated	Role-based, via resume upload
Workflow	Components studied apart	Ingestion to decision support

E. CODE QUALITY AND REPRODUCIBILITY

The repository separates concerns by folder: ingestion, processing (Spark), NLP, ML analysis and the dashboard each have their own module, and the dashboard code stays apart from the data-preparation scripts. Functions are small and named for what they do, key steps carry comments, and the project is under Git with a *.gitignore* that keeps virtual environments and bytecode out of the repository. A *requirements.txt* with minimum versions of pandas, NumPy, PySpark, PyArrow, Streamlit, pypdf, scikit-learn and Altair lets someone else rebuild the environment. We have not yet added automated tests, and exception handling in the scripts is minimal.

F. LIMITATIONS OF THE EVALUATION

Several caveats apply to everything above. The sample has 40 rows and ten unique descriptions. Extraction was never scored against human labels, so we can describe what the dictionary finds but cannot state its precision or recall. Since the descriptions list skills in plain comma-separated form, even a naive method does well here. We make no claims about runtime or throughput. Finally, the trend, emerging-skill and forecasting components could not be validated for lack of a time series.

IX. GAP ANALYSIS, CHALLENGES AND FUTURE DIRECTIONS

A. GAP ANALYSIS

Table 5 sets what the prototype does today against what the full design needs. Most of the distance is in the parts that move the system from a script collection toward a platform: streaming, declarative quality rules, orchestration and, above all, evaluation.

TABLE 5. Summary of the gap analysis between the current prototype and the proposed platform.

Component	Prototype today	Gap to close
Ingestion	Batch CSV	Kafka streaming and API connectors
Quality	Dedup, null drop, trim	Declarative rules, schema and range checks, reports
Extraction	22-skill dictionary	Aliases, unseen skills, context (e.g. “Excel”)
Storage	Parquet files	Warehouse star schema, date partitioning
Emerging skills	Threshold heuristic	Growth rate over time windows
Forecasting	Not implemented	Time-series models on a longer history

Component	Prototype today	Gap to close
Evaluation	Descriptive counts	Labelled test set, precision and recall
Operations	Scripts run by hand	Airflow scheduling, Docker packaging

B. CHALLENGES

Four challenges stand out. First, **data access**: the richest sources are commercial portals whose terms often forbid scraping, so a legitimate feed (an API partnership, or public vacancy datasets) has to be arranged. Second, **language ambiguity**: skill names collide with ordinary words, appear under many aliases, and new ones arrive faster than any dictionary is updated. Third, **bias and representativeness**: online postings over-represent some sectors and cities, so conclusions about "the market" can drift from the real labour market. Fourth, **evaluation**: without an annotated set we cannot tell whether a change to the extractor is an improvement.

C. FUTURE DIRECTIONS

- Connect real data: public vacancy datasets first, then a Kafka-fed stream of postings.
- Build a small human-annotated test set and report precision, recall and F1 for the dictionary and for learned extractors.
- Add the alias table, category layer and context rules, then trial spaCy and transformer-based extraction.
- Replace the snapshot heuristic with growth-rate detection over monthly windows, and add the forecasting baselines of Section V-D.
- Land curated tables in PostgreSQL, orchestrate the pipeline with Airflow, and package it with Docker.
- Run a small user study of the resume skill-gap feature with students.

X. CONCLUSION

We set out to build something that takes the mess of online job postings and hands a learner or a planner a clear answer about which skills matter. This paper presented SkillSync AI as an architecture and reported on the part of it that already runs. The prototype ingests postings, cleans them with PySpark, stores them as Parquet, finds skills with a dictionary-based extractor, ranks demand and opportunity, and compares a resume against a role in a Streamlit dashboard. On a 40-posting sample it found 22 skills, put Python in four of every five postings, and showed role and salary patterns that look sensible even if they are too small to prove anything.

The honest summary is that the engineering skeleton is in place while the intelligence on top of it is thin. Streaming, a warehouse, learned extraction and forecasting remain to be built, and the whole system needs evaluation on real data with labelled ground truth. We think the layered design makes those next steps incremental rather than a rewrite, and that is the main point we would defend.

REFERENCES

[1] E. Senger, M. Zhang, R. van der Goot, and B. Plank, "Deep learning-based computational job market analysis: A survey on skill extraction and classification from job postings," in *Proc. 1st Workshop Natural Lang. Process. Hum. Resour. (NLP4HR)*, St. Julian's, Malta, Mar. 2024, pp. 1–15.

[2] K. C. Nguyen, M. Zhang, S. Montariol, and A. Bosselut, "Rethinking skill extraction in the job market domain using large language models," in *Proc. 1st Workshop Natural Lang. Process. Hum. Resour. (NLP4HR)*, St. Julian's, Malta, Mar. 2024, pp. 27–42.

[3] Z. Ao, G. Horváth, C. Sheng, Y. Song, and Y. Sun, "Skill requirements in job advertisements: A comparison of skill-categorization methods based on wage regressions," *Inf. Process. Manage.*, vol. 60, no. 2, Mar. 2023, Art. no. 103185.

[4] G. Tzimas, N. Zotos, E. Mourelatos, K. C. Giotopoulos, and P. Zervas, "From data to insight: Transforming online job postings into labor-market intelligence," *Information*, vol. 15, no. 8, p. 496, Aug. 2024.

[5] P. C. Menezes *et al.*, "SKORE: Skill-oriented extraction engine from job advertisements using large language models," *MethodsX*, vol. 16, Art. no. 103907, 2026.

[6] T. Oleš, "In-demand skills: A shield against automation—Evidence from online job vacancies," *J. Labour Market Res.*, vol. 60, Art. no. 5, 2026.

[7] E. Zeydan and J. Mangues-Bafalluy, "Recent advances in data engineering for networking," *IEEE Access*, vol. 10, pp. 34449–34496, 2022.

[8] M. Zaharia *et al.*, "Apache Spark: A unified engine for big data processing," *Commun. ACM*, vol. 59, no. 11, pp. 56–65, Nov. 2016.

[9] J. Dean and S. Ghemawat, "MapReduce: Simplified data processing on large clusters," *Commun. ACM*, vol. 51, no. 1, pp. 107–113, Jan. 2008.

[10] W. McKinney, "Data structures for statistical computing in Python," in *Proc. 9th Python Sci. Conf.*, 2010, pp. 56–61.

[11] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, Oct. 2011.

[12] J. Kreps, N. Narkhede, and J. Rao, "Kafka: A distributed messaging system for log processing," in *Proc. NetDB Workshop*, 2011, pp. 1–7.

SUDHAMSHA A (Roll No. 2420030753) is an undergraduate student at [Institution Name]. [Add one or two lines on academic background and research interests.]

V AKSHAYA (Roll No. 2420090087) is an undergraduate student at [Institution Name]. [Add one or two lines on academic background and research interests.]

TANISHKA (Roll No. 2420030619) is an undergraduate student at [Institution Name]. [Add one or two lines on academic background and research interests.]

DR. N. SHIRISHA is an Associate Professor at [Institution Name] and guided this work. [Add qualifications and research interests.]